ROS 2 심화 미들웨어 2일차 보고서

Nov 8, 2026 · @동현

21기 예비단원 최동현 · 주제: 지연과 손실

1. 질문과 예상

질문: 와이파이에 지연과 손실을 걸면, 기본 QoS(RELIABLE)로 통신하는 ping/pong의 손실률, 평균 RTT, p95 RTT는 어떻게 바뀌는가?

실험 전 예상:

- 지연 100ms를 걸면 평균 RTT는 기준값보다 약 100ms 늘어날 것이다.
- RELIABLE은 빠진 메시지를 재전송하므로, 손실을 걸어도 측정 손실은 거의 0%일 것이다.
- 대신 재전송된 메시지는 늦게 도착하므로, 손실률이 클수록 p95가 커질 것이다.

2. 방법

RTT(Round Trip Time)는 메시지를 보낸 시각부터 그 응답이 돌아온 시각까지 걸린 시간이다. 가는 지연, 상대의 처리 시간, 오는 지연이 모두 더해진 값이다. 보낸 시각과 받은 시각을 같은 PC의 시계로 재므로 두 PC의 시계를 맞출 필요가 없다. 네트워크(와이파이)가 혼잡하거나 손실이 생기면 값이 커진다.

항목	내용
보내는 PC (ping)	LG gram 15Z90Q, Ubuntu 24.04, ROS 2 Jazzy, 기본 RMW(Fast DDS), 무선 `wlp0s20f3`
받는 PC (pong)	결과 1: 같은 PC / 결과 2-4: 같은 와이파이의 다른 PC
측정 코드	슬라이드의 ping.py / pong.py (`pingpong` 패키지, `std_msgs/String`)
QoS	기본값: RELIABLE, KEEP_LAST 10, VOLATILE
측정	200개, 초당 10번, 마지막 송신 후 2초 대기. p95 = 정렬한 값의 `v[int(0.95*(n-1))]`
ROS_DOMAIN_ID	결과 1은 5, 결과 2-4는 0

지연과 손실은 보내는 PC의 무선 인터페이스에 netem으로 걸었다. netem은 **내 PC에서 나가는 방향에만** 걸리므로, /ping만 늦어지거나 버려지고 돌아오는 /pong은 영향을 받지 않는다. 따라서 RTT에는 지연이 한 번만 더해진다.

```
# 지연 100ms ±20ms, 손실 5% 주입
sudo tc qdisc add dev wlp0s20f3 root netem delay 100ms 20ms loss 5%
# 손실만 20%로 바꾸기 (이미 걸려 있으므로 change)
sudo tc qdisc change dev wlp0s20f3 root netem delay 100ms 20ms loss 20%
# 실험 후 해제
sudo tc qdisc del dev wlp0s20f3 root
```

3. 결과: 기준값

결과 1. 같은 PC 안에서 ping과 pong 실행

손실 0.0%, 평균 4.8ms, p95 5.9ms다. 평균과 p95의 차이가 약 1ms로 지터가 작은 안정적인 상태다. 다만 같은 PC 안에서는 메시지가 와이파이를 거치지 않고 공유 메모리나 루프백으로 간다. 그래서 netem 실험의 기준값으로는 쓰지 않는다.

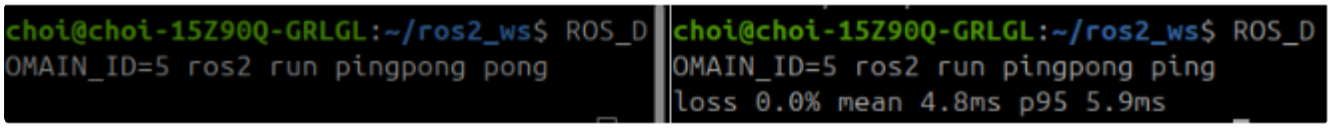


그림 1. 같은 PC 안에서 실행한 결과 (ROS_DOMAIN_ID=5)

결과 2. 다른 PC와 와이파이로 실행

같은 와이파이를 쓰는 다른 컴퓨터에서 pong을, 내 컴퓨터에서 ping을 실행했다. 손실 4.0%, 평균 5.1ms, p95 7.2ms다. netem 실험도 같은 경로(두 PC, 와이파이)로 했으므로 **이 값을 기준값으로 삼는다.**

평균과 p95는 결과 1과 비슷하지만 손실이 4% 생겼다. RELIABLE은 빠진 메시지를 재전송하므로, 이 4%는 네트워크 손실이라기보다 다른 원인으로 추정한다. 슬라이드 코드는 pong과 매칭되기 전부터 송신을 시작한다. discovery가 끝나기 전에 보낸 앞쪽 메시지는 받을 쪽이 없어 버려진다. 4%는 200개 중 8개, 약 0.8초 분량이다. 와이파이에서는 같은 PC보다 discovery가 오래 걸리므로 이런 손실이 생길 수 있다.

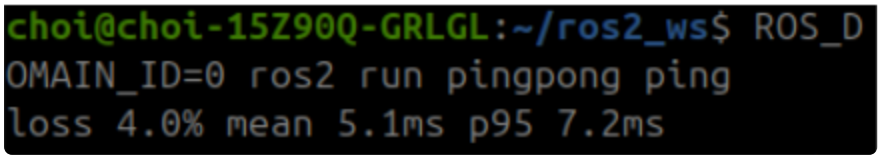


그림 2. 다른 PC와 와이파이로 실행한 결과 (ROS_DOMAIN_ID=0)

4. 결과: 지연과 손실 주입

결과 3. 지연 100ms ±20ms, 손실 5%

손실 7.0%, 평균 229.3ms, p95 784.5ms다. 기준값(5.1ms)에 지연 100ms만 더해지면 평균은 약 105ms여야 한다. jitter ±20ms는 개별 메시지가 흔들리는 폭이라 평균에서는 상쇄된다. 그런데 실제 평균은 예상보다 약 124ms 더 크게 나왔다.

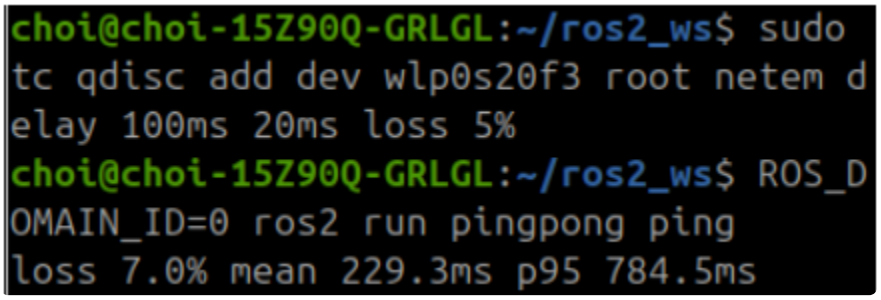


그림 3. 손실 5% 주입 결과

결과 4. 지연 100ms ±20ms, 손실 20%

손실 7.0%, 평균 372.7ms, p95 924.9ms다. 손실률을 높이자 평균과 p95가 모두 더 커졌다. 특히 p95는 평균의 약 2.5배로, 일부 메시지가 매우 늦게 도착했다.

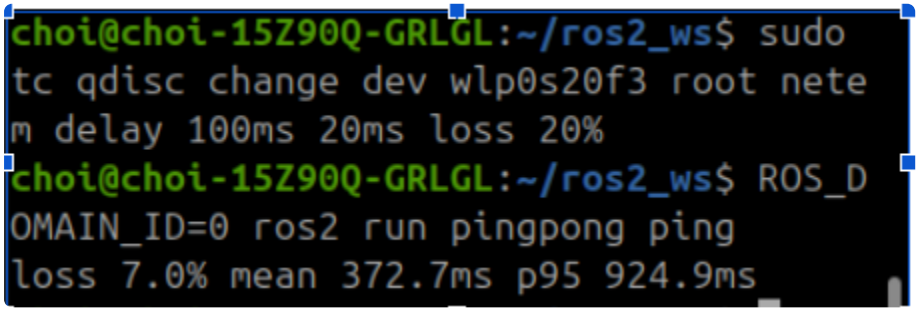


그림 4. 손실 20% 주입 결과

결과 요약

조건	경로	netem	손실	평균 RTT	p95 RTT
결과 1	같은 PC	없음	0.0%	4.8 ms	5.9 ms
결과 2 (기준)	두 PC, 와이파이	없음	4.0%	5.1 ms	7.2 ms
결과 3	두 PC, 와이파이	delay 100ms 20ms, loss 5%	7.0%	229.3 ms	784.5 ms
결과 4	두 PC, 와이파이	delay 100ms 20ms, loss 20%	7.0%	372.7 ms	924.9 ms

5. 해석

평균이 +100ms보다 훨씬 많이 늘어난 이유

RELIABLE QoS는 빠진 메시지를 재전송한다. 재전송된 메시지는 netem 지연을 한 번 더 거치므로 수백 ms 늦게 도착한다.

그런데 재전송만으로는 수치가 설명되지 않는다. 손실된 5%만 300ms씩 늦었다면 평균은 약 15ms만 늘어야 한다. 실제로는 124ms가 , 손실되지 늘었으므로않은 메시지들까지 함께 늦어졌다는 뜻이다.

RELIABLE은 메시지를 순서대로 전달한다. 하나가 빠지면 이미 도착한 뒤 메시지들도 그 메시지가 재전송되어 올 때까지 기다린다(head-of-line blocking). 초당 10번 보내므로 재전송에 0.5초가 걸리면 손실 1번에 약 5개의 메시지가 함께 늦어진다. 그래서 평균과 p95가 크게 늘었고, 손실률이 20%로 오르자 더 크게 늘었다.

RELIABLE인데 손실 7%가 남은 이유

재전송한다면 손실은 0%에 가까워야 한다. 원인 후보는 세 가지다.

1. 매칭 전에 송신한 메시지: discovery가 끝나기 전에 보낸 메시지는 받을 쪽이 없어 버려진다.
2. 짧은 대기 시간: 마지막 송신 후 2초만 기다리므로, 그 뒤에 도착한 응답은 손실로 집계된다.
3. 얕은 히스토리: KEEP_LAST 10이라 재전송되기 전에 오래된 메시지가 히스토리에서 밀려날 수 있다.

손실 5%와 20%에서 측정 손실이 똑같이 7%였다. netem 손실률과 상관없는 고정 비용이 있다는 뜻이므로, 1번(시작할 때 생기는 손실)에 무게가 실린다. 기준값에서 생긴 4%도 같은 원인으로 볼 수 있다. 다만 결과 2–4는 ROS_DOMAIN_ID=0(기본값)으로 측정해서, 같은 와이파이의 다른 사람 토픽과 섞였을 가능성도 있다.

6. 결론

기본 QoS(RELIABLE)는 손실을 재전송으로 복구하는 대신 그만큼을 지연으로 바꿨다. 손실률이 5%에서 20%로 오르자 평균 RTT는 229ms에서 373ms로, p95는 785ms에서 925ms로 늘었다. 그 대가는 평균보다 p95 같은 꼬리 지연에서 더 크게 나타났다.

예상과 달랐던 점

- 지연을 100ms 걸었는데 평균은 약 224ms 늘었다. 재전송과 순서 보장(head-of-line blocking)을 예상하지 못했다.
- RELIABLE인데도 손실이 0%가 아니라 7%였고, 손실률을 바꿔도 7%로 같았다.

실패한 시도

시도	무엇이 잘못됐나	배운 것
`ros2 pkg create ... --dependencies rclpy std_msgs pingpong`	`--dependencies`가 뒤 인자를 전부 의존성으로 받아 패키지 이름이 빠짐	패키지 이름을 옵션보다 앞에 쓴다
`ros2 run pingpong pong` → `NameError: rclpy`	파일에 `main()`만 넣고 import와 클래스를 빠뜨림	실행 파일은 모듈 전체를 불러온다
두 번째 `tc qdisc add` → `Exclusivity flag on`	이미 netem이 걸린 상태에서 다시 add	바꿀 때는 `change` 또는 `replace`
도메인 5와 0을 섞어서 측정	측정 조건이 달라짐	모든 측정에서 ROS_DOMAIN_ID를 고정하고 기록한다
`ros2 topic hz`의 window가 13에서 15로 건너뜀	손실로 오해할 뻔함. 1초 출력 주기와 발행 주기가 어긋난 것	손실은 seq 번호로 센다

아직 모르는 것

RELIABLE인데도 손실 7%가 생긴 정확한 원인(매칭 전 송신분인지, 대기 시간이 짧아서인지)을 확인하지 못했다.